

Laboratorio de Semáforos

Programación Concurrente y Paralela

Los archivos a completar están en `template-alumnos/`, marcados con `// TODO` en el lugar donde hay que agregar sincronización. Todos los ejercicios se resuelven con `java.util.concurrent.Semaphore`.

Cada archivo tiene su propio `main`, así que se compilan y corren de forma independiente, parados en la carpeta `template-alumnos/`.

Volcán Lanín

Un grupo de porteños viajó a Neuquén a subir el volcán Lanín. Por seguridad, e inexperiencia (son porteños), nadie puede arrancar un tramo del camino hasta que el resto del grupo llegó a la pirca (hito de piedras) que marca el final del tramo. De esa forma, nadie se adelanta solo, y nadie se queda perdido atrás sin que el resto se dé cuenta.

Parte 1: la pareja

(Para resolver en común)

Del grupo, llegó una pareja primero, Andrea y Bernardo, ya que el resto se perdió en el desierto pampeano. Como tienen algo de experiencia en la montaña, decidieron arrancar el camino solos, pero respetando la regla de esperarse en cada pirca que marca el final de un tramo.

Modelar la situación considerando que Andrea y Bernardo son threads separados y el camino tiene 5 tramos.

Archivo a completar: `template-alumnos/ExcursionPareja.java`

Compilar y ejecutar:

```
cd template-alumnos
javac ExcursionPareja.java
java ExcursionPareja
```

Parte 2: cardumen de porteños

Ahora sí, llegó el grupo, y apenas llegaron, arrancaron a subir. Realmente hay demasiados porteños, no sabemos muy bien cuántos hay. Por eso, desde el grupo de guardaparques nos encargaron de diseñar una solución a prueba de porteños, es decir, que sea flexible a grupos de N personas.

Construir una solución al problema de encuentro que sea escalable a un número arbitrario de porteños, permitiéndoles subir al Lanín con seguridad.

TIP: Los porteños están medio alterados de vivir en la ciudad y son tan ansiosos que a veces llegan a un tramo, bajan corriendo al principio del tramo y vuelven a recorrer el mismo tramo antes que llegue uno de sus compañeros. Su solución tiene que evitar que bajen corriendo porque es peligroso, y porque haría que contemos incorrectamente los porteños que hicieron un tramo.

Archivo a completar: `template-alumnos/MainExcursionGrupal.java`

Compilar y ejecutar:

```
cd template-alumnos
javac MainExcursionGrupal.java
java MainExcursionGrupal
```

Zorros grises de Saldungaray

Año electoral, año de arreglar rutas. La provincia de Buenos Aires decidió arreglar el tramo de la ruta provincial 72 cercano a la localidad de Saldungaray. Por eso, desde la municipalidad de Tornquist se enviaron policías de tránsito para regular el tránsito sobre el desvío temporal mientras se está arreglando la ruta. Lo malo es que la ruta permite el tránsito en una sola dirección. Los zorros grises (policías de tránsito) nos piden ayuda para modelar el problema y poder coordinar el paso de los vehículos, procurando dar paso a la mayor cantidad sin dejar esperando para siempre a alguna de las direcciones.

Parte 1: Ghostbusters

Por la noche, los únicos autos que transitan la ruta son autos fantasmas que vienen a visitar el cementerio de Saldungaray. Los autos son muy respetuosos de las leyes de tránsito, y respetan el trazado de la ruta y no pueden andar de frente a otros fantasmas. Sin embargo, se pueden pasar entre sí, es decir, un auto que viene de más atrás más rápido puede *atravesar* a uno que venía más lento adelante.

Completar el código para regular el tránsito de autos fantasmas por las rutas, asegurando que ningún auto se quede esperando más de $<N>$ (completar en base al generador) segundos.

Archivo a completar: `template-alumnos/CaminoUnaVia.java`

Hay que completar los métodos `entrar(direccion)` y `salir(direccion)`, además de las variables de sincronización necesarias.

Compilar y ejecutar:

```
cd template-alumnos
javac CaminoUnaVia.java
java CaminoUnaVia
```

Parte 2: Soja directa

De día, el camino lo recorren autos físicos. Modelar ahora el problema teniendo en cuenta que los autos no pueden chocar, es decir, un auto que entró al desvío después no puede salir antes de uno que entró antes. Además que el puente soporta como máximo K autos al mismo tiempo. Al igual que antes, ningún vehículo se puede quedar esperando más de $<N>$ (completar en base al generador) segundos.

Archivo a completar: `template-alumnos/CaminoUnaViaFIFO.java`

Al igual que en la parte 1, hay que completar los métodos `entrar(direccion)` y `salir(direccion)` y las variables de sincronización, ahora agregando el orden FIFO dentro de cada sentido.

Compilar y ejecutar:

```
cd template-alumnos
javac CaminoUnaViaFIFO.java
java CaminoUnaViaFIFO
```

Lollapalozers

Una empresa de eventos masivos y venta de tickets para nada monopólica hace un evento en Argentina con artistas locales e internacionales. Por culpa del malvado estado regulador no se pueden vender infinitos tickets, resulta que hay “*derechos del consumidor*” y “*normas de seguridad*”. No se deja libre al mercado. Por eso, tenemos que diseñar un sistema que permita vender la mayor cantidad de tickets posible, pero sin superar el total permitido de cada tipo.

- Mientras nadie compre un ticket, muchos posibles compradores pueden ver los tickets disponibles.
- Cuando un comprador decide comprar un ticket, no se puede ver la cantidad de tickets disponibles hasta que el comprador termine de efectuar la compra.
- Un comprador solo puede iniciar la compra de un ticket cuando la última actualización de la cantidad disponible que leyó coincide con lo actual. Si no, tiene que volver a leer la cantidad y chequear que puede efectuar la compra.

Archivo a completar: `template-alumnos/VentaTickets.java`

Hay que completar el método `comprar(tipo)` (con las variables de sincronización que hagan falta) y, en `main`, crear y arrancar un thread por comprador que llame a `comprar(tipo)` las veces que corresponda, guardando cuántos tickets logró comprar cada uno.

Compilar y ejecutar:

```
cd template-alumnos
javac VentaTickets.java
java VentaTickets
```